
Algoritmos Imunológicos

Algoritmos imunológicos

Algoritmos imunológicos são uma classe de algoritmos computacionais inspirados no sistema imunológico biológico. Eles simulam os mecanismos de defesa do sistema imunológico para resolver problemas de otimização, busca, aprendizado de máquina e reconhecimento de padrões.

Exemplos:

1. Algoritmo de seleção clonal

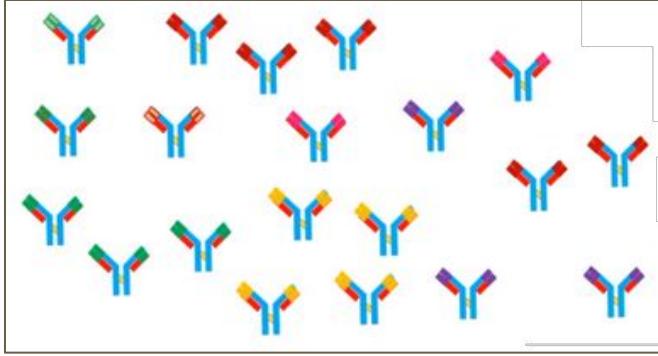
2. Algoritmo de seleção negativa (simula o processo de seleção negativa, onde os linfócitos T que reagem a células do próprio corpo são eliminados para evitar ataques autoimunes)

3. Algoritmos de células dendríticas (simula o comportamento das células dendríticas, que coletam informações sobre o ambiente e iniciam respostas imunes.)

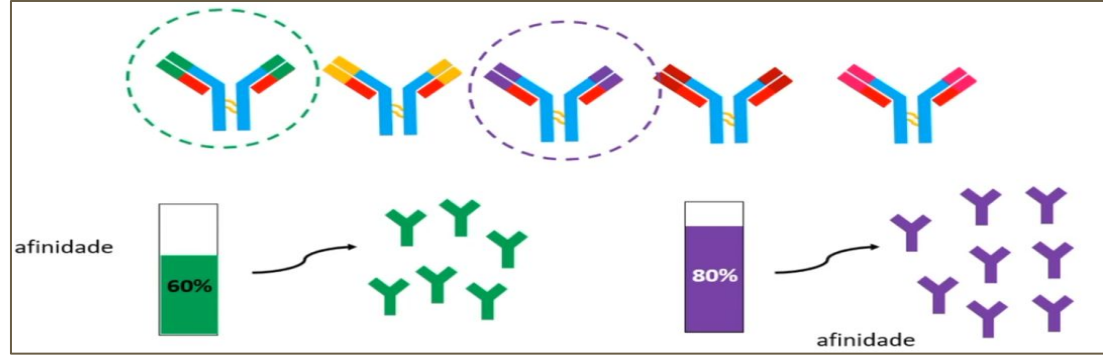
Algoritmo de seleção clonal

- O **algoritmo de seleção clonal** é um algoritmo imunológico inspirado no processo de seleção clonal que ocorre no sistema imunológico biológico.
- Usado para resolver **problemas de otimização e busca**, simulando a resposta imune a um antígeno.
- Simula a seleção clonal, onde os **anticorpos mais aptos são clonados e sofrem mutações para melhorar sua afinidade com o antígeno**.
- Similar ao processo de evolução darwiniana que ocorre com populações de indivíduos.

Algoritmo de seleção clonal



Anticorpos
produzidos pelo
sistema imunológico



Os melhores anticorpos serão selecionados.

Quanto maior a afinidade do anticorpo selecionado em relação ao antígeno, maiores as chances de reproduzir.

Algoritmo de seleção clonal



80

0,17

5 clones



72

0,16

5 clones



98

0,24

7 clones

→ Três anticorpos selecionados

→ Afinidade do anticorpo em relação ao antígeno

→ Afinidade (em porcentagem)

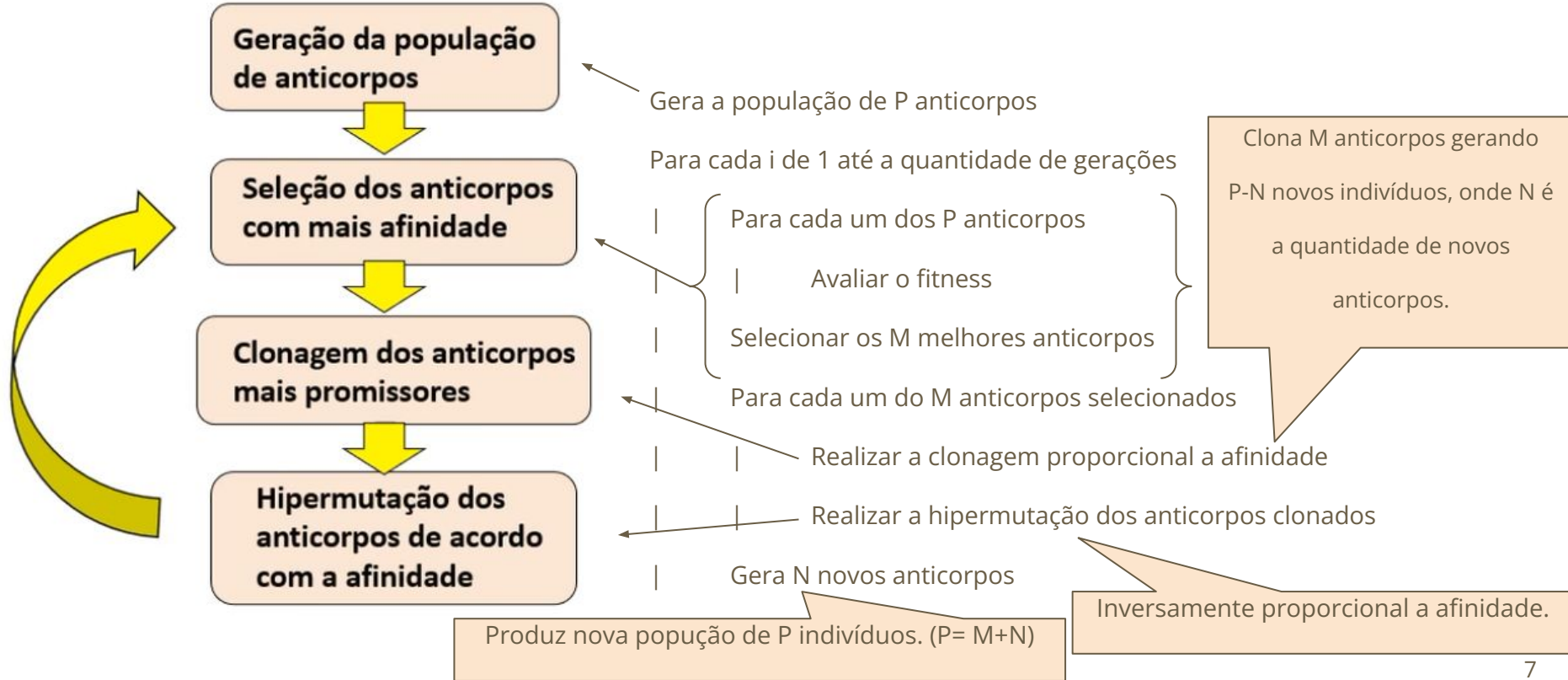
→ Quantidade de indivíduos gerados a partir da hipermutação. A quantidade de indivíduos gerados por cada anticorpo é proporcional à afinidade.

Os três anticorpos irão gerar a população de 17 indivíduos, de acordo com as quantidades mostradas na figura.

Algoritmo de seleção clonal

- Os 17 novos anticorpos gerados pela **clonagem** + **hipermutação** serão agrupados com outros 3 anticorpos **novos**, gerando uma nova população de 20 anticorpos.
- O processo se repete até que seja atingido um critério de parada, ou seja, a nova população passará pelos processos de **seleção** (melhores), **clonagem** e **hipermutação**.

Algoritmo de seleção clonal



Algoritmo de seleção clonal

Gera a população de P (20) anticorpos

Para cada i de 1 até a quantidade de gerações

| Para cada um dos P (20) anticorpos

| | Avaliar o fitness

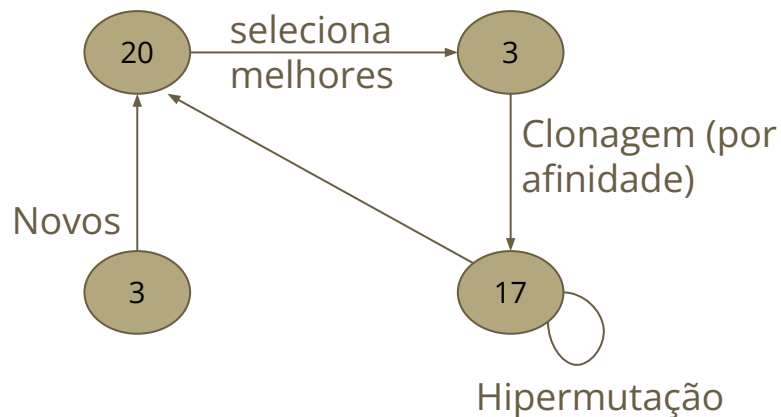
| Selecionar os M (3) melhores anticorpos

| Para cada um do M (3) anticorpos selecionados

| | Realizar a clonagem proporcional a afinidade

| | Realizar a hipermutação dos anticorpos clonados (17)

| Gera N (3) novos anticorpos



Algoritmo de seleção clonal

O algoritmo CLONALG para resolver tarefas de otimização pode ser descrito como segue:

1. **Inicialização:** criar uma população inicial de anticorpos (P).
2. **Avaliação de aptidão:** determinar a aptidão de cada elemento de P.
3. **Seleção e expansão clonal:** selecionar os n_1 elementos de maior aptidão de P e gerar clones desses anticorpos proporcionalmente à sua aptidão: quanto maior a aptidão, maior o número de cópias.
4. **Maturação por afinidade:** mutar todas essas cópias com uma taxa que é inversamente proporcional à sua aptidão: quanto maior a aptidão, menor a taxa de mutação. Adicionar esses indivíduos mutados à população P.
5. **Metadinâmica:** substituir um número n_2 de indivíduos de baixa aptidão por novos indivíduos (gerados aleatoriamente).
6. **Ciclo:** repetir os Passos 2 a 5 até que um certo critério de parada seja atingido.

Algoritmo de seleção clonal: clonagem

Gera a população de P anticorpos

Para cada i de 1 até a quantidade de gerações

| Para cada um dos P anticorpos

| | Avaliar o fitness

| Selecionar os M melhores anticorpos

| Para cada um do M anticorpos selecionados

| | Realizar a clonagem proporcional a afinidade

| | Realizar a hipermutação de P-N anticorpos clonados

| Gera N novos anticorpos

$$QC_k^i = \left(\frac{af_k^i}{\sum_{k=1}^n af_k^i} \right) * cl$$

QC_k^i = Quantidade de clones gerados para cada anticorpo k na interação i

af_k^i = Afinidade do anticorpo k na interação i

n = Num. de melhores anticorpos selecionados

$\sum_{k=1}^n af_k^i$ = Somatório das afinidades de todos os anticorpos selecionados

cl = Num. Total de clones a serem gerados

Algoritmo de seleção clonal: clonagem

$$QC_k^i = \left(\frac{af_k^i}{\sum_{k=1}^n af_k^i} \right) * Cl$$

QC_k^i = Quantidade de clones gerados para cada anticorpo k na interação i

af_k^i = Afinidade do anticorpo k na interação i

n = Num. de melhores anticorpos selecionados

$\sum_{k=1}^n af_k^i$ = Somatório das afinidades de todos os anticorpos selecionados

Cl = Num. Total de clones a serem gerados

Total de anticorpos: 20

Selecionados: 3

Clones gerados: 17

Afin anticorpo 1: 80

Afin anticorpo 2: 72

Afin anticorpo 3: 98

Algoritmo de seleção clonal: clonagem

$$QC_k^i = \left(\frac{af_k^i}{\sum_{k=1}^n af_k^i} \right) * Cl$$

QC_k^i = Quantidade de clones gerados para cada anticorpo k na interação i

af_k^i = Afinidade do anticorpo k na interação i

n = Num. de melhores anticorpos selecionados

$\sum_{k=1}^n af_k^i$ = Somatório das afinidades de todos os anticorpos selecionados

Cl = Num. Total de clones a serem gerados

Total de anticorpos: 20

Selecionados: 3

Clones gerados: 17

Afin anticorpo 1: 80

Afin anticorpo 2: 72

Afin anticorpo 3: 98

$$QC_k^i = \left(\frac{80}{80+72+98} \right) * 17$$

$$QC_k^i = 5,44 = 5$$

Algoritmo de seleção clonal: clonagem

$$QC_k^i = \left(\frac{af_k^i}{\sum_{k=1}^n af_k^i} \right) * Cl$$

QC_k^i = Quantidade de clones gerados para cada anticorpo k na interação i

af_k^i = Afinidade do anticorpo k na interação i

n = Num. de melhores anticorpos selecionados

$\sum_{k=1}^n af_k^i$ = Somatório das afinidades de todos os anticorpos selecionados

Cl = Num. Total de clones a serem gerados

Total de anticorpos: 20

Selecionados: 3

Clones gerados: 17

Afin anticorpo 1: 80

Afin anticorpo 2: 72

Afin anticorpo 3: 98

$$QC_k^i = \left(\frac{80}{80+72+98} \right) * 17$$

$$QC_k^i = 5,44 = 5$$

$$QC_k^i = \left(\frac{72}{80+72+98} \right) * 17$$

$$QC_k^i = 4,89 = 5$$

$$QC_k^i = \left(\frac{98}{80+72+98} \right) * 17$$

$$QC_k^i = 6,66 = 7$$

Algoritmo de seleção clonal: clonagem

$$QC_k^i = \left(\frac{af_k^i}{\sum_{k=1}^n af_k^i} \right) * Cl$$

QC_k^i = Quantidade de clones gerados para cada anticorpo k na interação i

af_k^i = Afinidade do anticorpo k na interação i

n = Num. de melhores anticorpos selecionados

$\sum_{k=1}^n af_k^i$ = Somatório das afinidades de todos os anticorpos selecionados

Cl = Num. Total de clones a serem gerados

Total de anticorpos: 20

Selecionados: 3

Clones gerados: 17

Afin anticorpo 1: 80

Afin anticorpo 2: 72

Afin anticorpo 3: 98

$$QC_k^i = \left(\frac{80}{80+72+98} \right) * 17$$

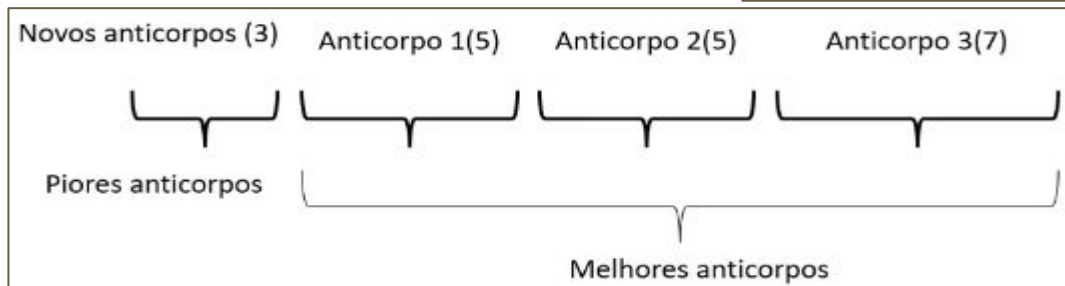
$$QC_k^i = 5,44 = 5$$

$$QC_k^i = \left(\frac{72}{80+72+98} \right) * 17$$

$$QC_k^i = 4,89 = 5$$

$$QC_k^i = \left(\frac{98}{80+72+98} \right) * 17$$

$$QC_k^i = 6,66 = 7$$



Algoritmo de seleção clonal: hipermutação

Gera a população de P anticorpos

Para cada i de 1 até a quantidade de gerações

Para cada um dos P anticorpos

Avaliar o fitness

Selecionar os M melhores anticorpos

Para cada um do M anticorpos selecionados

Realizar a clonagem proporcional a afinidade

Realizar a hipermutação de P-N anticorpos clonados

Gera N novos anticorpos

$$\text{hiperAnt}_k^i = (1 - (\text{afin}_k^i / \text{afin}_{\max})) * \beta$$

hiperAnt_kⁱ: taxa de hipermutação do anticorpo k na iteração i

afin_kⁱ: afinidade do anticorpo k na iteração i

afin_{max}: afinidade máxima

β: fator de mutação (ex. 0.1, 1, 10, etc)

Inversamente proporcional à afinidade.

Algoritmo de seleção clonal: hipermutação

$$\text{hiperAnt}_k^i = (1 - (\text{afin}_k^i / \text{afin}_{\max})) * \beta$$

hiperAnt_kⁱ: taxa de hipermutação do anticorpo k na iteração i

afin_kⁱ: afinidade do anticorpo k na iteração i

afin_{max}: afinidade máxima

β: fator de mutação (ex. 0.1, 1, 10, etc)

Afinidade = 79

β = 0,1

$$\text{hipAnt}_k^i = 1 - (79/121) * 0,1$$

$$\text{hipAnt}_k^i = 0,0347$$

Afinidade = 110

β = 0,1

$$\text{hipAnt}_k^i = 1 - (110/121) * 0,1$$

$$\text{hipAnt}_k^i = 0,009$$

Algoritmo de seleção clonal: exemplo

Objetivo: reconhecer o dígito representado em pixels, como na figura a direita

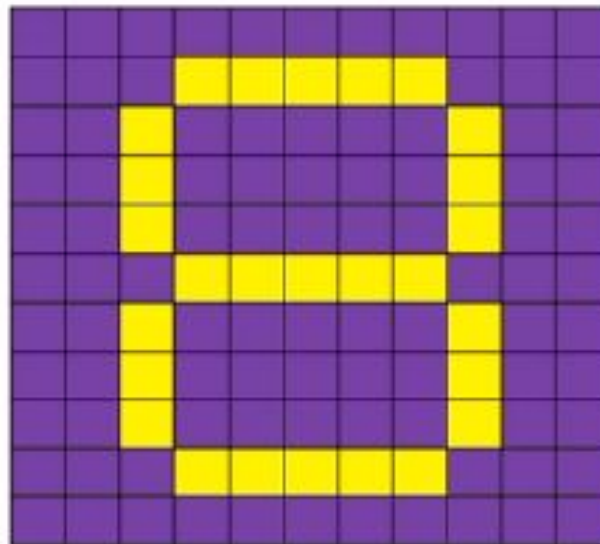
A figura com o número 8 em pixels (antígeno do problema) é representado por um vetor ou lista com vários valores de **0** e **1**, assim como os antígenos (11x11)

[0,0,0,0,1,0,0,1 ... 0,1,1,0,0]

1

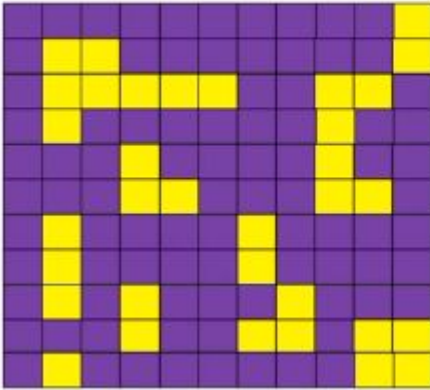
121

Antígeno

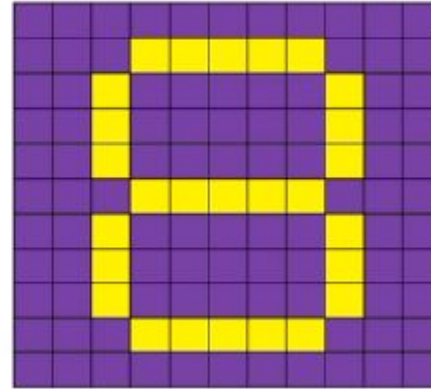


Algoritmo de seleção clonal: exemplo

Anticorpo



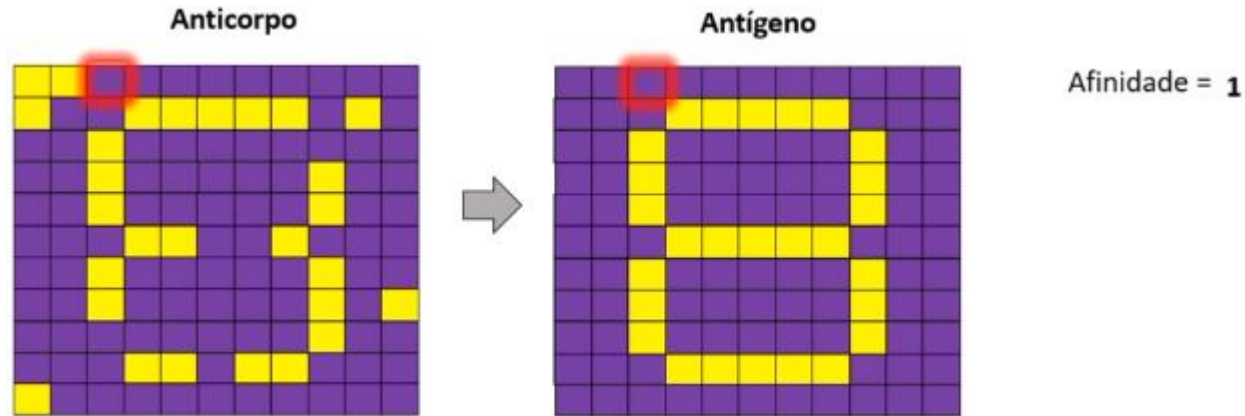
Antígeno



Algoritmo de seleção clonal: exemplo



Algoritmo de seleção clonal: exemplo



Afinidade máxima = 121

Estudar a implementação do CLONALG (Jupyter notebook).

Exercício

1. (V) ou (F) No CLONALG, os anticorpos representam soluções candidatas.
2. (V) ou (F) A afinidade de um anticorpo mede o quão bem ele reconhece o antígeno (resolve o problema).
3. (V) ou (F) A clonagem gera cópias exatas dos anticorpos.
4. (V) ou (F) A hipermutação introduz variações aleatórias nos genes dos clones.
5. (V) ou (F) A taxa de mutação na hipermutação é diretamente proporcional à afinidade do clone.
6. (V) ou (F) O CLONALG usa um mecanismo de seleção para escolher os melhores clones.
7. (V) ou (F) Os anticorpos com menor afinidade são mantidos na população para garantir a diversidade.
8. (V) ou (F) O CLONALG pode ser usado para resolver problemas de otimização.
9. (V) ou (F) O CLONALG garante encontrar a solução ótima global em todos os problemas.

Exercício

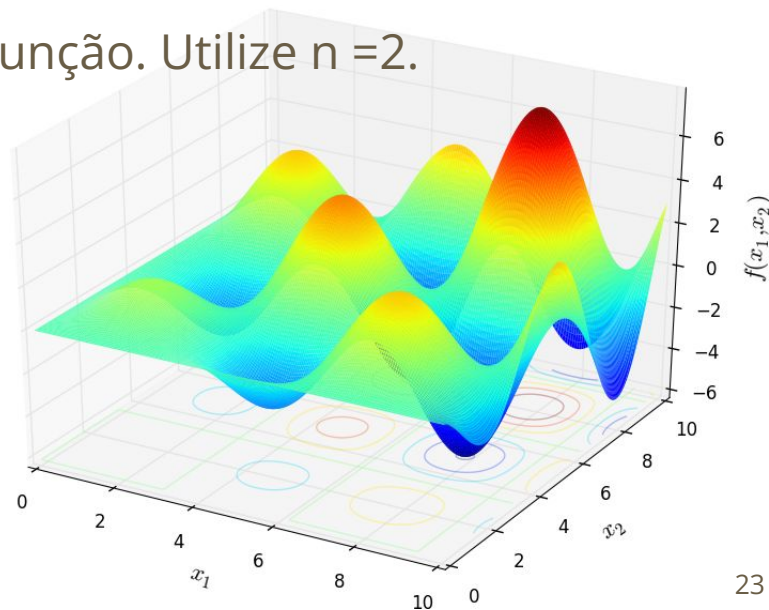
1. (V)
2. (V)
3. (V)
4. (V)
5. (F) A taxa de mutação na hipermutação é inversamente proporcional à afinidade do clone.
6. (V)
7. (F)
8. (V)
9. (F)

Trabalho prático 1 (não fazer)

Implemente o algoritmo imunológico Clonalg para resolver um problema de maximização multimodal da seguinte função

$$f_{\text{Alpine02}}(\mathbf{x}) = \prod_{i=1}^n \sqrt{x_i} \sin(x_i)$$

- n representa o número de dimensões da função. Utilize $n = 2$.
- $\mathbf{x} \in [0, 10]$
- Máximo Global da função é igual a $f(\mathbf{x}^*) = 2.808^n$.
 - Para $n=2$, o máximo é de 7.88 em $\mathbf{x}^* = (7.917, \dots, 7.917)$

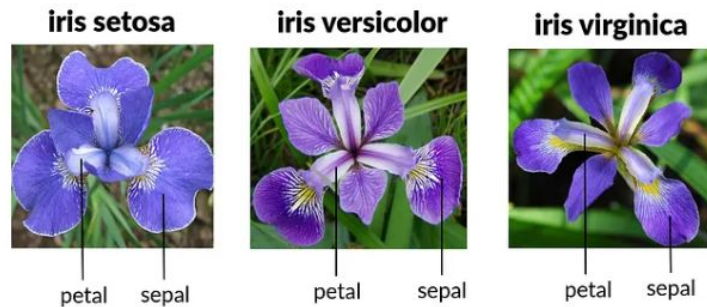


Trabalho prático 2

Utilizar algoritmos imunológicos para solucionar o problema de classificação de flores do tipo Iris.

Utilize o dataset “iris” disponível no pacote scikit-learn do Python.

O dataset possui 150 amonstras de flor, sendo 50 amostras de cada uma das três espécies de Iris



	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

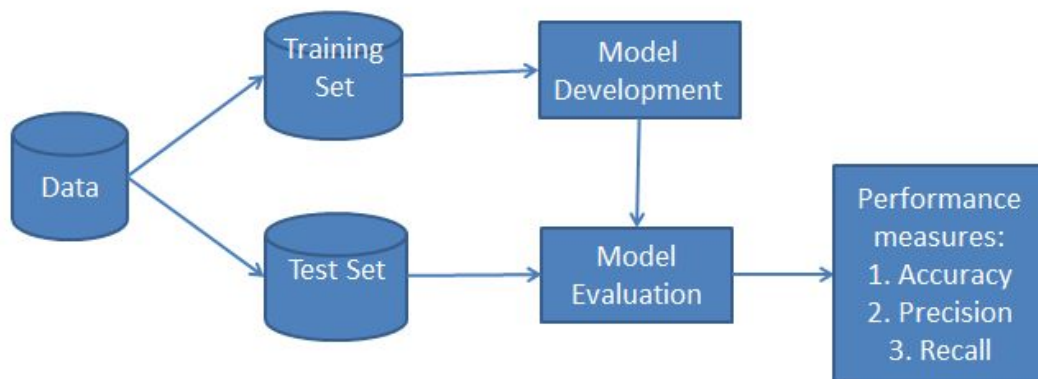
Trabalho prático 2

```
iris = load_iris()
X = iris.data
y = iris.target
n_features = 4
n_classes = 3
```

Divida o dataset em treino e teste. Ajuste os parâmetros do modelo usando o dataset de treino e realize a avaliação com o dataset de teste.

Dividir em conjunto de treinamento e teste

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, shuffle=True, random_state=42)
```



Trabalho prático 2

Você deve decidir como modelar o problema (o que representa cada anticorpo? quais os parâmetros do anticorpo? como será implementada a função de avaliação?)

A população de anticorpos deve ser de no máximo 50 indivíduos.

Exiba como a acurácia no conjunto de treino (média de acertos no conjunto de teste) varia no decorrer das gerações. Ao final, exiba a acurácia no conjunto de teste.

Analise qual o impacto do tamanho da população na acurácia. Varie o tamanho da população entre 10 e 50 e analise como isso interfere no valor da acurácia. Utilize 50 gerações. Apresente o resultado na forma de gráfico.

Você pode gerar dois códigos, um para exibir a acurácia ao longo da evolução e outro para analisar o impacto do tamanho da população na acurácia.

Trabalho prático 2

